Отчёт по лабораторной №1 по курсу “Методы вычислений”
Прямые методы решения СЛАУ

Выполнил:
Студент группы R4196
Панин Иван Михайлович

Преподаватель: Андреев Юрий Александрович

Санкт-Петербург, 2026

Основное задание

1. Сгенерировать квадратную вещественную матрицу и столбец свободных членов системы линейных алгебраических уравнений $Ax = b$ размерности $n = 6$.
2. Реализовать простой метод Гаусса, метод Гаусса с выбором ведущего элемента по столбцам, метод Гаусса с выбором ведущего элемента по всей матрице.
3. Решить систему методами, перечисленными выше. Решить систему с помощью любой библиотеки линейной алгебры. Сравнить результаты.
4. Найти обратную матрицу системы решая подсистемы системы $AX = E$, модифицировав соответствующим образом метод Гаусса.
5. Найти обратную матрицу системы с помощью любой библиотеки линейной алгебры. Сравнить результаты.
6. Вычислить число обусловленности в 1-й, 2-й и бесконечной норме. Охарактеризовать чувствительность данной системы к погрешностям исходных данных.

1-3. Метод Гаусса.
В рамках данной лабораторной работы мною были реализованы три разновидности метода Гаусса на языке программирования C, а именно: классический метод Гаусса, метод Гаусса с выбором ведущего элемента по столбцам и метод Гаусса с выбором ведущего элемента по всей матрице.

Для проверки работоспособности реализованных методов, им на вход подавались одинаковые СЛАУ. По результатам проверки, решения выбранных СЛАУ каждым из методов совпадали друг с другом, равно как и с решениями, полученными от заведомо правильных решателей.
После проверки работоспособности, мною было проведено сравнение скорости работы реализованных методов. Также, скорость работы реализованных методов была сопоставлена с алгоритмом Гаусса, написанном мной на языке ассемблера на 4 курсе в рамках дисциплины „Вычислительная математика“.

В рамках сравнения сравнения, методы решали одинаковую СЛАУ с 40 неизвестными. Методы gaus_classic, gauss_column_pivot и gauss_full_pivot были собраны с флагом оптимизации -O3.

Таблица сравнения скорости работы реализованных методов:

gauss_classic	gauss_column_pivot	gauss_full_pivot	gauss(asm)
6 мкс	6 мкс	15 мкс	4 мкс

Довольно просто понять, что самым медленным будет gauss_full_pivot, т.к. выбор ведущего элемента по всей матрице требует дополнительных вычислений.

Стоит отметить, что gauss_classic, равно как gauss_column_pivot, оба сходятся к выполнению за 6 мкс. Это обусловлено тем, что gauss_column_pivot почти не требует дополнительных вычислений в сравнении с gauss_classic. Тем не менее, с увеличение размеров исходной СЛАУ, будет увеличиваться отставание gauss_column_pivot от gauss_classic ввиду

необходимости переставлять потенциально большие строки матрицы. Тем не менее, данная проблема могла бы быть решена изменением представления матрицы в памяти, например задействовать указатели, чтобы дёшево менять местами строки.

Для того, чтобы понять, из-за чего версия метода Гаусса, написанная вручную на языке ассемблера, работает быстрее версий, созданных компилятором языка C, взглянем на код, сгенерированный компилятором.

Стоит отметить вычисление адреса внутри матрицы формы matrix[i * n + j], часто встречающееся в коде на языке C. В рамках реализации метода Гаусса на ассемблере я демонстрирую, как можно избежать наивного подсчёта адреса элемента матрицы с дорогой операцией умножения с помощью применения итераторов и последовательного прохождения матрицы. Как можно видеть, компилятор «не додумался» до данной оптимизации, поэтому использует наивный подход для вычисления адреса с относительно дорогостоящей инструкцией умножения.

```
mov    ebp,r14d
not    ebp
mov    edi,ebx
not    ebx
mov    DWORD PTR [rsp-0x68],ebp
movsxd rsi,ebp
imul   edi,eax
lea    r12,[rsi*4+0x0]

lea    r13,[rsi*4-0x4]

movsxd r9,edi
lea    rdx,[rcx+r9*1]
add    rax,r9
lea    r8,[r15+rdx*4]
movsxd rdx,ebx
lea    r9,[r15+rax*4]
lea    rbx,[rdx*4+0x0]
```

Далее стоит отметить оптимизации, применённые компилятором, задействующие SSE расширение процессоров x86_64:

```
shr     r11d,0x2
shl     r11,0x4
xchg    ax,ax
movups  xmm0,XMMWORD PTR [rdx+r10*1]
movups  xmm5,XMMWORD PTR [rsi+r10*1]
add     r10,0x10
mulps   xmm0,xmm5
addss   xmm1,xmm0
movaps  xmm2,xmm0
shufps  xmm2,xmm0,0x55
addss   xmm2,xmm1
movaps  xmm1,xmm0
unpckhps xmm1,xmm0
shufps  xmm0,xmm0,0xff
addss   xmm1,xmm2
addss   xmm1,xmm0
cmp     r11,r10
jne     1f00 <gauss_classic+0x280>
```

Можно заметить, что компилятор применил упакованные операции для оптимизации вычислений, что бывает довольно редко. Тем не менее, компилятор использует SSE инструкции, в то время как использование AVX2 инструкций с кодировкой VEX могло бы значительно снизить размер конечного кода, равно как позволило бы обработать больше упакованных данных, что в свою очередь, позволило бы повысить скорость работы алгоритма.

Версия алгоритма Гаусса, реализованная мною на языке ассемблера не использует операцию умножения в рамках нахождения адреса элемента матрицы и использует AVX2 VEX кодированные инструкции для обработки как скалярных, так и

упакованных данных, что и приводит к продемонстрированному выше выигрышу в производительности.

4-5. Нахождение обратной матрицы.
Для нахождения обратной матрицы достаточно применить метод Гаусса к матрице AX=E. После приведения исходной матрицы к единичному виду, на месте единичной матрицы будет обратная исходной. Данный алгоритм был реализован мной на языке программирования C и по сути своей является модификацией метода gauss_column_pivot, реализованного ранее.

6. Нахождение чисел обусловленности в 1й, 2й и бесконечной нормах.

Для нахождения числа обусловленности в первой норме, необходимо найти максимальные суммы элементов по столбцам в исходной и обратной матрицах, после чего умножить их.

Для нахождения числа обусловленности в бесконечной норме, необходимо найти максимальные суммы элементов по строкам в исходной и обратной матрицах, после чего умножить их.

Для нахождения числа обусловленности во второй норме, необходимо найти максимальное собственное число матрицы $B=A^TA$, после чего извлечь из него квадратный корень.

Мною были реализованы методы norm_1, norm_2 и norm_inf для нахождения соответствующих чисел обусловленности. Для реализации norm_2 пришлось реализовать функцию транспонирования матрицы, функцию умножения матриц и функцию поиска максимального собственного числа матрицы степенным методом.

По результатам проверки работоспособности реализованных методов, для тестовой матрицы было получено число обусловленности во второй норме = 15.6. Число обусловленности 15.6 указывает на очень хорошую устойчивость системы. При

решении такой системы во второй норме точность результата будет практически совпадать с точностью входных данных. Потеря точности составит около десятичных знака (log10(15.6)), что для типа float (7 знаков) является отличным результатом.

Заключение

В рамках данной лабораторной работы я ознакомился с различными вариациями метода Гаусса для решения СЛАУ. Также я познакомился с понятиями 1й, 2й и бесконечной норм.

Весь код, написанный в рамках данной лабораторной работы, равно как и код, написанный мною на 4 курсе на предмете «Вычислительная математика», предоставлен в открытом репозитории на github: https://github.com/adivanced-ui/MV_lab1.